

SQL grammar extensions since 2026-01-01

Scope: `parse/src/main/antlr4/sql/SQLSelectParser.g4` (and lexer tokens in the same file) on branch `Spring-2026-Extensions`, compared to the last pre-2026 snapshot (`f49203f`, 2025-12-31).

Purpose: Quick outline of **new or materially expanded** parse coverage—by statement/clause—not walker, generator, or test work unless the grammar itself changed.

Build artifacts (2026-08-09): `mvn package` in `parse/` produces:

Artifact	Role
<code>target/pss-parse-5.1.3-1.jar</code>	Main parser + runtime classes
<code>target/pss-parse-5.1.3-1-tests.jar</code>	Test utilities (<code>classifier: tests</code>)
<code>target/pss-parse-5.1.3-1-fat.jar</code>	Standalone jar-with-dependencies (Python/JS callers)

Script and entry points

- **script / sql_statement** — New top-level: semicolon-separated mixes of DDL, DML, WITH ... pipelines, and bare `query_expression`.
- **Dedicated endpoints** — Formal start rules for `update_end_point`, `delete_end_point`, `truncate_end_point` (update existed loosely before; delete/truncate endpoints are new).
- **literal_value** — Fragment endpoint wired for numeric/unsigned literals (walker key `LITERAL`).
- **predicand_value** — Unchanged name; `predicand/expression` surface expanded (see Expressions).

DDL

- **Full DDL primaries** — `ddl / ddl_primary` with `CREATE / DROP / ALTER / TRUNCATE` (not only `CREATE TABLE AS`).
- **CREATE object types** — Table (column list + options tail, or `AS query`), index, view, materialized view, function, procedure, macro, sequence, schema, database, role, user, stage, file format.
- **DROP / ALTER** — `ddl_object_type` + object name + optional tails.
- **Opaque option capture** — `generic_ddl_paren_content` (nested parens) and `generic_ddl_options` (verbatim tail to `;` or EOF); options are **not** fully decomposed in the grammar.
- **TRUNCATE dialect split** — Snowflake (`TRUNCATE TABLE name`) vs Postgres (optional `TABLE`, multiple names).

WITH (CTE)

- **CTE body shapes** — query under a CTE may be `query_expression`, **INSERT**, **UPDATE**, **DELETE**, or **VALUES** (not only `SELECT`).
- **Substitution CTE body** — `query_alias variable_identifier` (tuple/query substitution as CTE body).
- **Nested WITH** — `cte_body` allows `with_query` (Snowflake-style nested `WITH` inside a CTE only).
- **DML + outer WITH** — Same `with_query` wrapper used for leading CTEs on DML statements (per comment blocks in grammar).

INSERT

- **Postgres path as default** — `insert_expression` → `postgres_insert` (Snowflake insert remains `snowflake_insert` building block).
 - **ON CONFLICT** — `conflict_target`, `DO NOTHING`, `DO UPDATE SET` + optional `WHERE` (lexer keywords `CONFLICT / DO / NOTHING` with Identifier fallback when tokenized as names).
 - **RETURNING** — Shared returning rule on insert (Postgres-style).
 - **Target table rule** — `insert_target_table_primary` excludes table-function call shape from `INSERT INTO ... (cols)`.
 - **Sources** — `query_expression`, substitution, `VALUES`, `DEFAULT VALUES`.
 - **Targets** — `db_object_name`, substitutions, **Jinja/DBT** `jinja_identifier`.
-

UPDATE

- **update_end_point** — First-class parse entry.
 - **Clause options** — `SET` assignment list, optional `FROM`, `WHERE`, **RETURNING** (Postgres/Snowflake overlap; reuses `select_list`).
 - **Table resolution** — Uses `table_primary` / left-factored aliasing (see `FROM`).
-

DELETE

- **Snowflake vs Postgres** — `delete_snowflake_expression` (no `RETURNING`) vs `delete_postgres_expression` (requires `delete_returning`).
 - **USING** — `delete_using_clause` with `table_reference_list`.
 - **CTE validity** — Grammar comments distinguish which variant may appear as a CTE data source.
-

TRUNCATE

- **Standalone** — `truncate_end_point` and `truncate_statement_primary` under DDL/script.
 - **Dialect variants** — See DDL (Snowflake vs Postgres).
-

SELECT / set operations

- **query_primary removed** — Set-op members use `set_operation_member`: `subquery` | `query_specification` | `variable_identifier` (avoids dead `query_primary` alternative).
 - **QUALIFY** — Snowflake post-window filter: `qualify_clause` after `HAVING`, before `ORDER BY`.
 - **Set ops** — `UNION / EXCEPT / INTERSECT` with optional `DISTINCT/ALL` on qualifiers (structure retained; member rules updated).
-

FROM / joins / relation sources

- **LATERAL** — `lateral_modifier` on comma and join operands.
- **Relation sources** — `table_source_primary` adds **table functions**, **VALUES**, **Jinja**, substitutions; `jinja_identifier` in `FROM/INSERT targets`.
- **PIVOT / UNPIVOT** — Postfix `table_relational_modifier` on `table_primary` (see below).

- **Alias factoring** — Aliases on `table_primary` / modifier aliases gated so PIVOT/UNPIVOT do not admit ambiguous double-optional AS (left-factoring change).
 - **Join extension** — `join_extension` substitution variable still supported; `join_extension_primary` for snippet endpoint.
-

PIVOT / UNPIVOT (Snowflake)

- **UNPIVOT** — Value/name columns, IN list, INCLUDE NULLS / EXCLUDE NULLS.
 - **PIVOT** — Aggregate clause (single aggregate or Snowflake comma list: AVG/COUNT/MAX/MIN/SUM), FOR column, IN content:
 - literal value list (with optional AS prefix labels),
 - ANY [ORDER BY ...],
 - subquery.
 - **DEFAULT ON NULL (...)** — Optional pivot tail.
 - **Shared IN-list helpers** — `relational_modifier_*` rules shared between pivot and unpivot.
-

Table functions (Snowflake-oriented)

- **table_function_primary** — TABLE(...) wrapper or bare call.
 - **Named functions** — FLATTEN (incl. LATERAL_FLATTEN name), GENERATOR, RESULT_SCAN, INFER_SCHEMA, VALIDATE, plus **generic** identifier(...) fallback.
 - **Named arguments** — INPUT=>, PATH=>, ROWCOUNT=>, LOCATION=>, etc., with semantic predicates on argument lists where required.
 - **table_argument_literal** — String/numeric args for TF kwargs (T2.1 coverage).
-

GROUP BY / HAVING

- **GROUP BY ALL** — `groupby_all_option`.
 - **GROUP BY DISTINCT** — `groupby_distinct_body` wrapping grouping elements.
 - **GROUPING SETS** — First-class `grouping_sets_list` / `grouping_element` alternative (baseline had ROLLUP/CUBE/empty/parenthesized sets but not explicit GROUPING SETS grouping element).
 - **Legacy** — Still allows `GROUP BY select_list` alternative.
-

WHERE / predicates

- **quantified_comparison_predicate** — Wired on predicate (= ALL|ANY|SOME (subquery)); rule existed pre-2026 but was not on the main predicate chain in the same way.
 - **substitution_predicate** — Tuple/substitution in boolean context.
 - **exists_predicate** — EXISTS on subquery or substitution.
 - **Removed** — `unique_predicate` (SQL UNIQUE (subquery) —not Snowflake).
 - **LIKE / ILIKE** — `like_any_predicate` (LIKE ANY / ILIKE ANY) with optional ESCAPE.
-

Expressions, casts, functions

- **Unparenthesized arithmetic in predicands** — `row_value_predicand` includes `common_value_expression` (+, *, etc. without extra parens).
 - **Postgres :: casts** — Chained `(expr)::type`, including `NULL::type`; predicand and value-expression paths updated.
 - **CAST / inline casts** — Broader NULL and chain casting support in value expressions.
 - **String / search functions** — `POSITION` (incl. SQL IN form), `INSTR`, `CHARINDEX` (2- and 3-argument shapes).
 - **Snowflake scalars** — `IFF`, `MD5`, `REVERSE` (lexer + routine names).
 - **TRIM** — Dropped unreachable `trim_operands` alternative; MySQL-style and `FROM` forms retained.
 - **JSON path columns** — Colon path segments on `column_reference` / `column_primary` (variant access).
-

EXTRACT / DATE_PART

- **extract_expression** — Expanded `extract_field` (Postgres + **Snowflake** field tokens, case-insensitive `EXTRACT` lexer).
 - **extract_source** — Column, datetime literal, **interval**, broader predicand sources; dedicated **predicand** endpoint coverage.
 - **date_part_expression** — Snowflake/Postgres `DATE_PART(field FROM source)` with case-insensitive `DATE_PART` lexer (new keyword placed after `PIVOT` block to limit token renumbering).
-

Substitutions, Jinja, DBT

- **Angle-bracket substitutions** — Expanded use in columns, `FROM`, CTE bodies, predicates, set-op members.
 - **jinja_identifier** — `{{ ref|source|stream|var|env_var(...) }}`, `config.get(...)`, `{{ this }}` / `{{ target.schema }}` chains, keyword args in calls.
 - **Insert/update symbol tables** — Grammar allows DML shapes that carry substitutions (walker work; grammar enables targets/sources).
-

VALUES

- **Script / CTE / FROM** — `values_statement_primary` as relation source.
 - **Insert** — `insert_values_statement` / matrix forms unchanged in spirit; better integrated with DML and CTE rules.
-

Lexer / grammar hygiene

- **Many new keywords** — DDL objects, DML (`DELETE`, `TRUNCATE`, `RETURNING`, conflict words), `PIVOT/UNPIVOT`, table-function args, Snowflake extract parts, `DATE_PART`, `SETS`, etc.
- **Keyword placement policy** — Comment in lexer: append late-keyword block before `Identifier` to avoid mass token ID churn in goldens.
- **Parser predicates** — Jinja recognition, disallowed join/set-op aliases, implicit alias gating, Snowflake TF mode literals, DDL not applicable here but same file.

Intentionally shallow or out of scope in the grammar

- **DDL interiors** — Captured as opaque text, not per-option rules (see `generic_ddl_*`).
- **Hive INSERT OVERWRITE** — Documented in comments; not a first-class alternate in current `insert_expression`.
- **Full Postgres INSERT** — `postgres_insert` builds on Snowflake-shaped core; some Postgres-only insert variants may still be comment-only.

Related non-grammar work (same period, for context only)

Walker AST, symbol tables, dialect warnings, JaCoCo exit coverage, golden test consolidation, and `SQLStatementGenerator` updates accompany these grammar changes but are not listed rule-by-rule here.